

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

«Реализация межсервисного взаимодействия посредством
очередей сообщений»

Выполнила:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить RabbitMQ и реализовать через него межсервисное взаимодействие между сервисами, полученными в ЛР2.

Тема проекта: вариант 1 – приложение для бронирования столиков в ресторанах. Работа выполнена поверх результата ЛР2 (три микросервиса: auth, restaurants, bookings + gateway).

Ход работы

1. Принятое решение

В обмене участвуют только два сервиса: **bookings** публикует события, **restaurants** их получает. auth и gateway очередь не используют, архитектура из ДЗ4 и ЛР2 в остальном не менялась.

Событиями сделала создание брони и смену её статуса – это те моменты, после которых клиенту не нужен немедленный ответ от другого сервиса, значит, их можно передавать асинхронно, не задерживая основной запрос.

Получатель не просто выводит событие в консоль, а сохраняет его в новую таблицу `notification_log` в `restaurants_db`. Решила так, потому что таблицу можно посмотреть в любой момент запросом к базе.

2. Дизайн новой архитектуры и работа очереди

Схема взаимодействия: **bookings** публикует сообщение в **exchange** (это точка входа внутри RabbitMQ, которая сама по себе сообщения не хранит), **exchange** по **routing key** раскладывает сообщения по **очередям** (спискам, где сообщения ждут обработки), а **restaurants** слушает свою очередь и забирает из неё сообщения по мере готовности. Параметры обмена приведены в таблице 1.

Таблица 1 – Элементы обмена

Элемент	Значение
Exchange	booking-events, тип direct, durable
Routing keys	booking.created, booking.status_changed
Очередь	restaurants.booking-notifications, durable, привязана к обоим routing key
Публикует	bookings-service
Получает	restaurants-service, пишет в notification_log

Формат сообщения – JSON:

```
{
  "event": "booking.created",
  "bookingId": 12,
  "restaurantId": 5,
  "userId": 7,
  "status": "PENDING",
  "occurredAt": "2026-10-05T10:05:00Z"
}
```

RabbitMQ подняла отдельным контейнером в `docker-compose.yml` (образ `rabbitmq:3-management`, он уже включает веб-панель управления). Порты выбрала нестандартные – **5673** для самого протокола и **15673** для панели.

Для каких ситуаций нужна именно очередь, а не обычный HTTP-запрос между сервисами, как в ЛР2: HTTP годится, когда без ответа другого сервиса операцию нельзя завершить – например, при создании брони нужно синхронно узнать у `restaurants`, существует ли столик, иначе бронь просто нечем будет проверить. Очередь годится, когда сервису важно сообщить факт, но результат этого сообщения ни на что не влияет прямо сейчас – "бронь создана", "статус изменился" не меняют ответ клиенту, их можно обработать позже, даже если получатель в этот момент не работает. Асинхронность в проекте есть ровно в двух местах – публикация `booking.created` и `booking.status_changed` из `bookings` – и появляется она в тот момент, когда `publish` вызван: `bookings` не ждёт, пока RabbitMQ доставит сообщение до `restaurants`, а сразу идёт дальше и отвечает клиенту.

3. Синхронное и асинхронное в одном сценарии

Создание и обработка одной брони на самом деле задействуют оба вида связи².

При создании брони `bookings` по-прежнему **синхронно** обращается к `restaurants`, чтобы проверить столик и слот – эта проверка осталась от ЛР2 и никак не связана с Д35: без неё нельзя решить, можно ли вообще создавать бронь. Только после того как бронь сохранена в базу, `bookings` **асинхронно** публикует событие `booking.created` и, не дожидаясь его доставки, отвечает клиенту.

Из этого следует важное следствие для проверки: если `restaurants` недоступен, создание брони упадёт с 503 ещё на синхронной проверке столика, до того как дело дойдёт до публикации события.

Поэтому демонстрировать асинхронность на создании брони бессмысленно — она в принципе не может пройти при выключенном `restaurants`, вне зависимости от того, как устроена очередь. Смена статуса брони устроена иначе: она проверяется по полю самой брони локально в базе `bookings` и `restaurants` не требует — значит, именно на ней видно, что публикация события не зависит от того, жив ли получатель в момент отправки.

Здесь есть ещё один нюанс: право на смену статуса тоже проверяется по-разному в зависимости от роли. Если статус меняет `RESTAURANT_ADMIN`, сервис всё равно обращается к `restaurants`, чтобы узнать, каким рестораном он управляет, — и такой запрос при выключенном `restaurants` тоже упал бы, но уже по причине проверки прав, а не из-за очереди. Поэтому для чистоты сценария статус меняла от лица `ADMIN`: у этой роли такой проверки нет, и запрос доходит до публикации события без единого обращения к `restaurants`.

Честная часть, которую стоит признать. Для самого частого в жизни случая — администратор именно этого ресторана подтверждает свою бронь — устойчивость к сбою `restaurants` на практике не наступает, потому что запрос падает раньше, на авторизации.

4. Что было сделано

1. **Добавила RabbitMQ в docker-compose.** Новый сервис `rabbitmq:3-management` в корневом `docker-compose.yml`, порты `5673` и `15673`, логин и пароль вынесла в `.env`.
2. **Поставила зависимость `amqp-lib`.** В `services/bookings` и `services/restaurants`; `auth` и `gateway` не трогала.
3. **Написала клиент очереди в bookings.** Подключение к RabbitMQ с несколькими попытками (ретраями), если брокер ещё не поднялся, и объявление `durable exchange booking-events`, и функцию `publish`.
4. **Написала клиент очереди в restaurants.** То же подключение, плюс объявление `durable-очереди restaurants.booking-notifications` с привязкой к обоим `routing key`, и функцию `consume`.
5. **Добавила публикацию событий в bookings.** `booking.created` — сразу после сохранения новой брони; `booking.status_changed` —

сразу после смены статуса. Оба вызова по принципу «выстрелил и забыл»: ошибка публикации только логируется и не прерывает ответ клиенту.

6. **Создала таблицу `notification_log`.** В `restaurants_db`, создаётся автоматически через `synchronize: true`, как и остальные таблицы этого сервиса.
7. **Написала обработчик входящих событий (consumer).** При получении сообщения сохраняет запись в `notification_log`; при успешной записи подтверждает получение (`ack`), при сбое – `nack` без повторной постановки в очередь.
8. **Реализовала опциональный эндпоинт уведомлений.** `GET /restaurants/{id}/notifications` с тем же ограничением доступа, что и у остальных операций администратора ресторана – переиспользовала уже готовую проверку прав, а не писала новую.
9. **Не стала документировать новый эндпоинт в `openapi.yaml`.** Публичный API этим файлом менять нельзя по правилам проекта, а эндпоинт уведомлений в задание Д32 не входил.
10. **Приняла ограничение по обработке сбоя записи в базу.** При неудачной записи в `notification_log` сообщение отклоняется без повторной постановки и без отдельной очереди для таких случаев (dead-letter) – такое сообщение теряется, но задание этого не требовало, поэтому усложнять не стала.

5. Проверка

5.1 Обычный путь, без сбоев

Создала бронь через gateway (столик 4, слот 4, дата 09.10.2026) – ответ 201, `id: 7`. Сменила её статус на `CANCELLED` – ответ 200. Проверила таблицу `notification_log`:

id	event	booking_id	restaurant_id	received_at
4	booking.created	7	3	2026-09-22 18:20:06.859467+00
5	booking.status_changed	7	3	2026-09-22 18:20:14.090187+00

Ровно по одной записи на каждое событие – как и ожидалось.

5.2 Асинхронность: restaurants недоступен при смене статуса

Создала бронь при работающем restaurants (столик 5, слот 5, дата 10.10.2026) – 201, id: 8. Остановила процесс restaurants. Сменила статус этой брони на CONFIRMED от лица ADMIN – ответ **200**, а не 503, как и должно быть по причине, описанной в разделе 3. В панели RabbitMQ очередь restaurants.booking-notifications показала одно сообщение, ожидающее обработки (messages_ready: 1, consumers: 0). Запустила restaurants снова – через несколько секунд в notification_log появилась новая строка с event = booking.status_changed, а очередь вернулась к messages_ready: 0.

5.3 Устойчивость к сбою самого RabbitMQ

Остановила контейнер rabbitmq, оставив restaurants и bookings работающими. Создала бронь через gateway (столик 1, слот 1, дата 11.10.2026) – ответ **201**, id: 9: попытка публикации события упала внутри защиты от ошибок и не помешала созданию брони. В консоли bookings при этом появилась строка о том, что канал недоступен и событие не отправлено, но сам процесс не остановился. Запустила RabbitMQ обратно и проверила notification_log – строки с booking_id = 9 там нет: сообщение о создании этой брони потеряно безвозвратно, как и описано в разделе 4 как принятое ограничение.

5.4 Права доступа к эндпоинту уведомлений

Запросила GET /restaurants/1/notifications от лица администратора, который не назначен на этот ресторан, – ответ **403**. Затем под администратором именно этого ресторана – ответ **200**, список из записей по ресторану 1, включая события из предыдущих сценариев.

Все сценарии, кроме потери сообщения при сбое самого брокера (раздел 5.3, это принятое ограничение, а не дефект), отработали так, как и предполагалось при проектировании.

Вывод

В ходе работы к архитектуре из ЛР2 добавлена очередь сообщений RabbitMQ между bookings и restaurants: при создании и смене статуса брони bookings публикует событие, а restaurants обрабатывает его асинхронно и сохраняет в таблицу notification_log. Отдельно показано,

что в рамках одного и того же сценария бронирования уживаются оба вида межсервисной связи: синхронная проверка столика при создании брони, унаследованная от ЛР2, и асинхронная публикация события уже после того, как бронь сохранена, – и что продемонстрировать устойчивость к сбою нужно там, где связь действительно асинхронная, а не на операции, которая и так синхронно зависит от другого сервиса. При разработке найдена и исправлена гонка при старте restaurants между подключением к базе и запуском обработчика очереди. Далее эти же сервисы, включая RabbitMQ, будут упакованы в контейнеры в ЛР3.